SIMATS ENGINEERING
CO5-ASSESSMENT TOOL1-Design Task

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1709**

Register Number	**192312620**
Name	**V.Madhulika**

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications.* (BTL 5)

Assessment Tool Weightage: 40%

Time Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Code of Conduct:

I V.Madhulika(192312620) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

V.Madhulika
Signature of the student:

1. End-to-End System Design

Design a scalable AI-based system using **PySpark and RDD operations** to analyze real-time social media data and generate intelligent recommendations, including system architecture, data flow, and agent integration.

2. Distributed Intelligent Agent System

Develop a **multi-agent AI system architecture** that uses PySpark for distributed data processing and enables agents to collaborate in solving a large-scale optimization problem.

3. Big Data AI Pipeline Design

Design a complete **distributed AI pipeline** using RDD transformations for data preprocessing, feature engineering, and model training, ensuring scalability and fault tolerance.

4. Real-Time Decision-Making System

Construct an AI-based system integrating **RDD-based streaming data processing** with intelligent agents for real-time decision-making in applications such as smart traffic or healthcare.

5. Cloud-Based Distributed AI Application

Design a **cloud-enabled AI system** using PySpark and intelligent agent architectures that supports large-scale data analytics, dynamic resource allocation, and adaptive learning.

RUBRICS FOR CALCULATION

Evaluation Criteria	Problem Understanding & Requirement Analysis	System Architecture Design	Use of PySpark & RDD Operations	Intelligent Agent Integration	Scalability & Distributed Computing Design	Data Flow & Processing Pipeline	Innovation & Creativity	Clarity, Presentation & Documentation
Weightage	10%	20%	15%	15%	10%	10%	10%	10%

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)
Problem Understanding & Requirement Analysis	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect
System Architecture Design	Complete, scalable, well-structured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete
Use of PySpark & RDD Operations	Efficient and correct use of RDD transformations/ actions	Mostly correct usage	Limited or partial use	Incorrect/missing
Intelligent Agent Integration	Strong integration with clear agent roles and logic	Good integration	Basic integration	Poor/no integration
Scalability & Distributed Computing Design	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed
Data Flow & Processing Pipeline	Clear, logical, and well-defined data flow	Mostly clear	Some gaps	Poorly defined

Innovation & Creativity	Highly innovative, realistic, and impactful solution	Some innovation	Basic idea	No originality
Clarity, Presentation & Documentation	Very clear, neat diagrams, well-organized report	Mostly clear	Somewhat organized	Poor presentation

Question 1: End-to-End System Design - Real-Time Social Media Recommendation System

Problem Understanding

Design a scalable AI system that ingests real-time social media data (posts, likes, shares, comments), processes it using PySpark/RDD operations, and generates intelligent, personalised recommendations for each user.

System Architecture

Social Media Streams	Kafka Ingestion	Spark Streaming (Micro-batches)	RDD Transformations	Feature Store	Recommendation Agent	User Feed

- Data Ingestion Layer: social-media APIs / Kafka topics continuously publish raw posts, likes and shares.
- Stream Processing Layer: Spark Structured Streaming consumes each micro-batch and converts it into an RDD for the current window.
- Distributed Processing Layer (PySpark/RDD): text cleaning and tokenising (flatMap), stop-word removal (filter), word/topic counting and sentiment scoring (map + reduceByKey).
- Feature Store: aggregated per-user interest vectors persisted to HDFS/Parquet or a NoSQL store for low-latency lookup.
- Intelligent Agent Layer: a utility-based recommendation agent scores candidate posts/products against each user's interest vector (cosine similarity, recency, popularity).
- Serving Layer: top-ranked recommendations pushed to the user's feed via an API/message queue.
- Feedback Loop: clicks/likes on recommended content are captured as new streaming data, closing the loop so the agent keeps learning.

PySpark / RDD Design (illustrative)

```
stream = KafkaUtils.createDirectStream(ssc, ['posts_topic'], kafka_params)
posts   = stream.map(lambda msg: json.loads(msg[1]))
words   = posts.flatMap(lambda post: tokenize(post['text']))
clean   = words.filter(lambda w: w not in STOPWORDS_BC.value)
counts  = clean.map(lambda w: (w, 1)).reduceByKey(lambda a, b: a + b)
interest_vec = counts.map(lambda kv: build_user_interest(kv))
recs    = interest_vec.mapPartitions(lambda part: score_candidates(part, catalog_BC.value))
recs.foreachRDD(lambda rdd: push_to_feed(rdd.collect()))
```

Scalability & Fault Tolerance

- RDD lineage (DAG) lets Spark recompute only the lost partitions if a worker node fails, instead of restarting the whole job.
- Checkpointing of the streaming context guards against driver failure and unbounded lineage growth.
- Data is partitioned by user-id so recommendation scoring for different users runs fully in parallel across executors.
- Broadcast variables share the large, mostly-static content catalog to every executor once instead of shuffling it repeatedly.
- Dynamic resource allocation (YARN/Kubernetes) scales executors up during traffic spikes (e.g. trending topics) and down when idle.

Question 2: Distributed Intelligent Agent System - Multi-Agent Delivery Route Optimisation

Problem Understanding

Design a multi-agent architecture that uses PySpark for distributed data processing and allows agents to collaborate on a large-scale combinatorial-optimisation problem - minimising total travel cost across a citywide package-delivery network with many warehouses.

System Architecture

Delivery Data (RDD, partitioned by zone)	Zone Worker Agents (parallel)	Boundary Coordination (broadcast/accumulators)	Global Aggregator Agent	Merged Global Route Plan

- Coordinator (Master) Agent: partitions the delivery region into zones and assigns one RDD partition per zone.
- Worker Agents (one per Spark partition/executor): each optimises delivery routes within its own zone in parallel using a local heuristic (e.g. nearest-neighbour, simulated annealing, or a small genetic algorithm) over its RDD partition of delivery points.
- Coordination Layer: Spark's driver acts as the message-passing backbone; boundary information between adjacent zones is exchanged through broadcast variables and accumulators so overlapping deliveries are not double-served.
- Global Aggregator Agent: on the driver, merges every zone's sub-solution into one global route plan, resolves boundary conflicts, and can trigger another optimisation round (iterative map-reduce refinement) until costs converge.

PySpark / RDD Design (illustrative)

```
points   = sc.parallelize(delivery_points)
zoned    = points.map(lambda pt: (zone_of(pt), pt)).partitionBy(NUM_ZONES)
local_solutions = zoned.mapPartitions(lambda zone_agent_optimize)
global_solution = local_solutions.reduce(merge_and_resolve_conflicts)
```

Scalability & Fault Tolerance

- Each zone agent's work is an independent RDD partition, so the number of zones/agents scales horizontally with cluster size.
- RDD lineage allows a failed worker's zone to be recomputed on another executor without restarting the whole optimisation.
- Iterative rounds are checkpointed periodically to avoid an ever-growing lineage graph slowing recovery.

Question 3: Big Data AI Pipeline Design - Distributed Preprocessing, Feature Engineering & Training

Problem Understanding

Design a complete distributed AI pipeline, built entirely from RDD transformations, that takes raw data through cleaning, feature engineering and model training - remaining scalable and fault-tolerant throughout.

Pipeline Architecture

Raw Data (HDFS/S3)	Cleaned RDD	Feature-Engineered RDD	LabeledPoint RDD	Distributed Model Training (MLlib)	Model Evaluation	Model Store

Stage	RDD Operations Used
Data Ingestion	sc.textFile()/newAPIHadoopFile() loads data from HDFS/S3, automatically partitioned for parallelism.
Data Cleaning	filter() drops nulls/corrupt records; map() casts types and normalises values.
Feature Engineering	flatMap()/map() for tokenisation and encoding; reduceByKey() for per-entity aggregation; join() to merge multiple feature-source RDDs.
Vectorisation	map() with a vector-assembler UDF produces a LabeledPoint RDD (features + label).
Model Training	MLlib distributed training (e.g. LogisticRegressionWithSGD) - gradients are computed per partition and combined with treeAggregate().
Evaluation	map() to pair predictions with true labels, then reduce() to compute aggregate metrics (accuracy/F1).
Persistence	model.save() writes the trained model to distributed storage for later serving.

Scalability & Fault Tolerance

- cache()/persist() intermediate RDDs (e.g. the cleaned RDD) that are reused across multiple downstream stages, avoiding recomputation.
- Prefer reduceByKey()/combineByKey() over groupByKey() to minimise shuffle volume across the cluster.
- repartition()/coalesce() rebalance data across executors to avoid stragglers on skewed partitions.
- RDD lineage (the DAG of transformations) means any lost partition - from cleaning through to training - can be automatically recomputed from source data, giving the whole pipeline fault tolerance without manual checkpoints, though periodic checkpointing is still used for very long lineages.

Question 4: Real-Time Decision-Making System - Smart Traffic Signal Control

Problem Understanding

Construct an AI system that combines RDD-based streaming data processing with intelligent agents to make real-time traffic-signal decisions at city intersections.

System Architecture

Traffic Sensors/Cameras	Kafka Stream	Spark Streaming (RDD per batch)	Per-Intersection Aggregation	Signal-Control Agent (per intersection)	Signal Controller

- Sensors: IoT loop sensors/cameras at each intersection continuously stream vehicle counts and speeds.
- Ingestion & Streaming: Kafka buffers the sensor stream; Spark Structured Streaming converts each micro-batch interval into an RDD.
- RDD Processing: map() computes a congestion index per road segment; reduceByKey() aggregates it per intersection; reduceByKeyAndWindow() maintains a rolling trend over the last few minutes.
- Signal-Control Agent (model-based reflex, one logical agent per intersection, computed in parallel via mapPartitions): perceives the current + recent congestion index and decides an action - extend green, switch phase, or hold - to maximise throughput and minimise wait time.
- Actuation: the chosen action is sent to the physical traffic-signal controller through an IoT gateway.
- Central Dashboard: aggregates city-wide congestion for monitoring and can push reroute suggestions to navigation apps.

PySpark / RDD Design (illustrative)

```
sensor_stream = KafkaUtils.createDirectStream(ssc, ['traffic_topic'], kafka_params)
readings = sensor_stream.map(lambda m: json.loads(m[1]))
congestion = readings.map(lambda r: (r['intersection_id'], r['vehicle_count']))
agg = congestion.reduceByKeyAndWindow(lambda a, b: a + b, windowDuration=300, slideDuration=30)
decisions = agg.mapPartitions(lambda part: signal_agent_decide(part))
decisions.foreachRDD(lambda rdd: send_to_signal_controllers(rdd.collect()))
```

Scalability & Fault Tolerance

- Each intersection's stream key is processed independently, so adding intersections/executors scales the system horizontally.
- Write-ahead logs and DStream checkpointing recover in-flight streaming state after a driver/executor failure.
- Kafka topic replication guarantees at-least-once delivery of sensor readings even if a broker fails.

Question 5: Cloud-Based Distributed AI Application

Problem Understanding

Design a cloud-enabled AI system built on PySpark and intelligent-agent architectures that supports large-scale data analytics, dynamic resource allocation, and adaptive (online) learning.

System Architecture

Cloud Object Storage (S3/ADLS)	Managed Streaming (Kafka/Event Hub)	PySpark on Kubernetes (RDD ETL)	Adaptive Learning Agents (microservices)	REST API / Serving Layer	Monitoring & Auto-scaling

- Cloud Infrastructure: a Kubernetes cluster (AWS/Azure/GCP) runs Spark, giving elastic, on-demand compute.

- Data Layer: cloud object storage (S3/ADLS) is the system of record; a managed streaming service (Kafka/Event Hub) handles real-time ingestion.
- Processing Layer: PySpark jobs perform RDD-based ETL and analytics with dynamicAllocation.enabled=true, so the executor pool grows/shrinks with workload.
- Intelligent Agent Layer: adaptive learning agents (recommendation, anomaly detection, etc.) run as cloud microservices, consuming freshly processed RDD batches via pub/sub and retraining periodically - an online-learning feedback loop.
- Serving & Monitoring: predictions are exposed through a REST API/cloud-functions layer; CloudWatch/Grafana-style monitoring tracks cluster health and job metrics; a data-drift agent triggers automated retraining (MLOps/CI-CD) when input distributions shift.

Scalability & Dynamic Resource Allocation

- Horizontal auto-scaling of Spark executors based on real-time queue/CPU load, including spot/preemptible instances for cost efficiency.
- Partition tuning and repartitioning keep work balanced as cluster size changes.
- Checkpointing to cloud storage lets jobs resume after an availability-zone failure without losing streaming state.
- The adaptive-learning loop (retrain-on-drift) lets the deployed models keep improving as new cloud data arrives, without manual redeployment.